

Atelier 13 navigateur 3

Kra

Matisse

Sommaire

objectif	1
I. Concepts Théoriques Abordés	1
1. Qu'est-ce qu'un événement ?	1
2. Les Gestionnaires d'événements (Handlers)	1
3. Les Objets Clés	1
II. Travaux Pratiques et Réalisations	2
1. Cacher un élément ("Cacher sur clic" & "Se cacher")	2
2. Gestion des coordonnées ("Le Ballon")	4
3. Menu Déroulant ("Menu Glissant")	7
4. Manipulation du DOM ("Bouton de fermeture")	9
5. Algorithme Complexe ("Le Carrousel")	11
III. Conclusion	13

objectif

Gestion des événements dans le DOM (Document Object Model) **Objectif** : Apprendre à rendre une page web interactive en réagissant aux actions de l'utilisateur (clics, mouvements de souris, chargement, etc.).

I. Concepts Théoriques Abordés

1. Qu'est-ce qu'un événement ?

Un événement est un signal émis par le navigateur pour indiquer qu'une action s'est produite.

- **Exemples courants** : `click` (souris), `keydown` (clavier), `submit` (formulaire), `DOMContentLoaded` (chargement de la page).

2. Les Gestionnaires d'événements (Handlers)

Pour réagir à un événement, on utilise une fonction appelée "Handler". Il existe trois façons de les assigner :

- **Attribut HTML (Déconseillé)** : On écrit le JS directement dans la balise (ex: `onclick="..."`).
 - *Limites* : Mélange le code et le design, difficile à maintenir.
- **Propriété DOM (Classique)** : On assigne une fonction à la propriété de l'élément (ex: `elem.onclick = function()`).
 - *Limites* : On ne peut assigner qu'un **seul** gestionnaire par événement. Une nouvelle assignation écrase la précédente.

- **Méthodes Spéciales (Recommandé - Standard Moderne)** : Utilisation de `addEventListener(event, handler)` et `removeEventListener`.
 - *Avantages* : Permet d'ajouter plusieurs gestionnaires sur le même événement et gère des options avancées.

3. Les Objets Clés

- **Le mot-clé `this`** : Dans un gestionnaire, `this` fait référence à l'élément HTML qui a déclenché l'événement.
- **L'objet `event`** : Le navigateur transmet automatiquement un objet (souvent nommé `event` ou `e`) à la fonction gestionnaire. Il contient les détails de l'action (coordonnées de la souris `clientX`, type d'événement, touche pressée, etc.).

II. Travaux Pratiques et Réalisations

Durant cet atelier, nous avons mis en pratique la théorie à travers 5 exercices progressifs :

1. Cacher un élément ("Cacher sur clic" & "Se cacher")

- **Objectif** : Modifier le style CSS via JS au clic.
- **Méthode** : Utilisation de `style.display = 'none'`.
- **Acquis** : Compréhension de l'utilisation de `this` pour qu'un bouton agisse sur lui-même sans avoir besoin de son ID.

Cacher sur clic

Ajouter JavaScript à la `button` pour faire `<div id="text">` disparaître quand on clique dessus.

```
1  <!DOCTYPE html>
2  <html lang="fr">
3  <head>
4      <meta charset="UTF-8">
5      <title>Exercice Cacher sur clic</title>
6  </head>
7  <body>
8
9      <input type="button" id="monBouton" value="Cliquez pour cacher le texte">
10
11      <div id="text">
12          Ceci est le texte magique qui va disparaître !
13      </div>
14
15      <script>
16          // ÉTAPE 1 : Sélection des éléments
17          // On récupère le bouton et la div par leurs ID
18          let bouton = document.getElementById('monBouton');
19          let texte = document.getElementById('text');
20
21          // ÉTAPE 2 : Assignment du Handler
22          bouton.onclick = function() {
23
24              // --- ÉTAPE 3 : Action ---
25              // On modifie le style pour cacher l'élément
26              // 'none' retire l'élément de l'affichage
27              texte.style.display = 'none';
28          };
29      </script>
30
31  </body>
32  </html>
```

- L'événement : Nous utilisons l'événement click, qui se produit quand la souris clique sur l'élément.
- Le Gestionnaire (Handler) : Comme expliqué dans la section "Propriété DOM", nous assignons une fonction anonyme (function() {...}) à la propriété elem.onclick.
- L'Objet Événement (Optionnel) : Bien que nous ne l'ayons pas utilisé ici, il faut se rappeler que le navigateur transmet automatiquement un objet event à cette fonction, qui contient des détails sur le clic (coordonnées, etc.).

Se cacher

Crée un bouton qui se cache tout seul au clic.

La particularité ici, c'est que l'élément qui déclenche l'action est le même que celui qui subit l'action.

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <title>Exercice Se Cacher</title>
6 </head>
7 <body>
8
9   <button id="boutonTimide">Clique-moi pour me cacher !</button>
10
11   <script>
12     // 1. On récupère l'élément
13     let monBouton = document.getElementById('boutonTimide');
14
15     // 2. On assigne la fonction au clic
16     monBouton.onclick = function() {
17       // 3. 'this' fait référence à l'élément sur lequel on a cliqué (monBouton)
18       this.style.display = 'none';
19     };
20   </script>
21
22 </body>
23 </html>
```

Dans cet exercice, l'utilisation de this est essentielle.

- Si on avait utilisé `document.getElementById('boutonTimide').style.display...` à l'intérieur de la fonction, le navigateur aurait dû rechercher l'élément une seconde fois.
- Avec `this`, le code est plus performant et générique : la fonction sait déjà qu'elle doit agir sur l'élément qui a déclenché l'événement.

Quels handlers exécutent?

Il y a un bouton dans la variable. Il n'y a aucun handlers dessus.

Quels handlers s'exécutent au clic après le code suivant? Quelles alertes apparaissent?

```
1 button.addEventListener("click", () => alert("1"));
2
3 button.removeEventListener("click", () => alert("1"));
4
5 button.onclick = () => alert(2);
```

Les deux alertes vont s'afficher ("1" puis "2")

Étape 1 : Ajout du premier gestionnaire

- `button.addEventListener(...)` ajoute une fonction anonyme (fléchée) qui affiche "1".
- Le navigateur stocke cette fonction en mémoire.

Étape 2 : Tentative de suppression (Le Piège)

- On essaie de retirer l'événement avec `RemoveEventListener`.

matisse kra
bts sio

- Le problème : La fonction `() => alert("1")` écrite à la ligne 3 est une nouvelle fonction, différente de celle de la ligne 1, même si le code à l'intérieur est identique. Ce sont deux objets différents en mémoire.
- Résultat : `removeEventListener` ne trouve pas cette nouvelle fonction dans la liste des écouteurs, donc il ne supprime rien. L'alerte "1" est toujours active

Étape 3 : Ajout du second gestionnaire (onclick)

- on utilise la propriété DOM `onclick`.
- Comme vu dans le cours, on peut définir des gestionnaires à la fois avec une propriété DOM (`onclick`) et avec `addEventListener`. L'un n'écasse pas l'autre s'ils sont définis via ces deux méthodes différentes.

2. Gestion des coordonnées ("Le Ballon")

- **Objectif** : Déplacer une image (`position: absolute`) à l'endroit précis du clic sur un terrain (`position: relative`).
- **Difficulté** : Calculer la position relative par rapport au terrain et non à la fenêtre entière.
- **Acquis** :
 - Utilisation de `event.clientX` et `event.clientY`.
 - Utilisation de `getBoundingClientRect()` pour gérer le défilement (scroll) de la page.
 - Logique algorithmique pour empêcher le ballon de sortir du cadre (collisions).

Déplacez le ballon à travers le terrain

Déplacez le ballon à travers le terrain jusqu'à un clic.

[ballon2.html](#)

Click on a field to move the ball there.



CSS : `#field` doit être en `position: relative` pour servir de repère (0,0) à ses enfants. `#ball` doit être en `position: absolute`.

JS : On utilise `getBoundingClientRect()` pour savoir où est le terrain sur l'écran, même si on a scrollé.

Centrage : L'exigence "Le centre de la balle devrait passer exactement sous le pointeur" oblige à soustraire la moitié de la taille de la balle.

```
1 <!DOCTYPE HTML>
2 <html> Add "lang" and/or "xml:lang" attributes to this "<html>" element
3
4 <head> Add a <title> tag to this page.
5 <meta charset="utf-8">
6 <style>
7   #field {
8     width: 200px;
9     height: 150px;
10    border: 10px solid black;
11    background-color: #00FF00;
12    overflow: hidden;
13
14    /* INDISPENSABLE : Pour que la balle se positionne par rapport au terrain */
15    position: relative;
16    cursor: pointer; /* Pour montrer qu'on peut cliquer */
17  }
18
19  #ball {
20    /* INDISPENSABLE : Pour pouvoir déplacer la balle */
21    position: absolute;
22
23    /* Initialisation */
24    left: 0;
25    top: 0;
26
27    /* Animation CSS lors du déplacement */
28    transition: all 1s ease-out;
29  }
30 </style>
31 </head>
32
33 <body style="height:200px">
34
35   Click on a field to move the ball there.
36   <br> The ball should never leave the field.
37
38   <div id="field">
39     
40   </div>
41
42   <script>
43     // Sélection des éléments
44     let field = document.getElementById('field');
45     let ball = document.getElementById('ball');
```

```
46
47   field.onclick = function(event) {
48
49     // 1. Où est le terrain par rapport à la fenêtre ?
50     // getBoundingClientRect() gère automatiquement le scroll
51     let fieldCoords = this.getBoundingClientRect();
52
53     // 2. Calcul des coordonnées
54     // event.clientX : position souris fenêtre
55     // fieldCoords.left : position gauche terrain fenêtre
56     // field.clientLeft : épaisseur bordure gauche (10px ici)
57     // ball.clientWidth / 2 : pour centrer la balle sous la souris
58
59     let ballTop = event.clientY - fieldCoords.top - field.clientTop - (ball.clientHeight / 2);
60     let ballLeft = event.clientX - fieldCoords.left - field.clientLeft - (ball.clientWidth / 2);
61
62     // 3. Empêcher la balle de sortir (Limites) [cite: 387]
63
64     // Limite HAUT
65     if (ballTop < 0) {
66       ballTop = 0;
67     }
68
69     // Limite GAUCHE
70     if (ballLeft < 0) {
71       ballLeft = 0;
72     }
73
74     // Limite DROITE
75     // field.clientWidth est la largeur intérieure (sans bordures)
76     if (ballLeft + ball.clientWidth > field.clientWidth) {
77       ballLeft = field.clientWidth - ball.clientWidth;
78     }
79
80     // Limite BAS
81     if (ballTop + ball.clientHeight > field.clientHeight) {
82       ballTop = field.clientHeight - ball.clientHeight;
83     }
84
85     // 4. Appliquer les nouvelles coordonnées
86     ball.style.left = ballLeft + 'px';
87     ball.style.top = ballTop + 'px';
```

1. La sélection des éléments (Variables)

- Concept : On crée des références vers les éléments HTML.
- Pourquoi ? Pour pouvoir les manipuler plus tard sans avoir à demander au navigateur de les rechercher à chaque fois (optimisation).

2. Le Gestionnaire d'Événement (La Fonction)

- Concept : C'est une fonction anonyme assignée à l'événement onclick.
- Le paramètre event : C'est crucial. Quand le clic se produit, le navigateur "injecte" automatiquement un objet contenant tous les détails du clic (notamment les coordonnées X et Y de la souris) dans cette variable event.

3. Le Repérage dans l'Espace (Mathématiques)

C'est la partie la plus technique.

- this : Fait référence à field (l'élément sur lequel on a cliqué).
- getBoundingClientRect() : Cette méthode renvoie la position exacte du terrain par rapport à la fenêtre visible du navigateur. C'est indispensable pour gérer le défilement (scroll) de la page. Si on scroll, les coordonnées changent, et cette fonction nous donne les valeurs à jour.

4. Le Calcul des Coordonnées (Algorithme)

Nous devons calculer ballLeft (position X locale) et ballTop (position Y locale).

La formule pour ballLeft :

Décortiquons ce calcul :

1. event.clientX : La position de la souris sur tout l'écran (ex: 500px).
2. - fieldCoords.left : On soustrait la distance entre le bord gauche de l'écran et le début du terrain. On ramène le repère à 0 (le coin du terrain).
3. - field.clientLeft : On enlève l'épaisseur de la bordure noire du terrain (10px). Sinon, la balle serait décalée de l'épaisseur du cadre.
4. - (ball.clientWidth / 2) : On enlève la moitié de la largeur de la balle.
 - Sans ça : Le coin haut-gauche de la balle serait sous ta souris.
 - Avec ça : Le centre de la balle est sous ta souris.

5. Les Limites (Logique Conditionnelle / Pseudo-code)

Pour empêcher la balle de sortir, on utilise des structures conditionnelles SI ... ALORS.

Pour le bord haut et gauche :

- Traduction : Si la position calculée est négative (la balle dépasse en haut), on la force à 0 (collée au bord).

Pour le bord bas et droit :

- La logique : On vérifie si la position gauche de la balle + sa largeur dépasse la largeur intérieure du terrain (field.clientWidth).
- L'action : Si oui, on place la balle exactement à la fin du terrain moins sa propre largeur.

6. L'Application (Mise à jour du DOM)

- Concept : On modifie les propriétés CSS en direct.
- Important : En CSS, les positions ont besoin d'une unité. On doit donc faire une concaténation (ajouter le texte 'px' à la fin du nombre calculé).

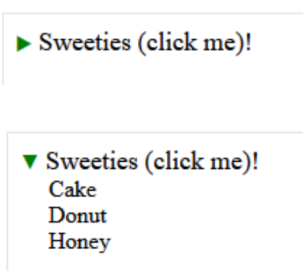
matisse kra
bts sio

3. Menu Déroulant ("Menu Glissant")

- **Objectif** : Créer un menu qui s'ouvre et se ferme avec une animation de flèche.
- **Méthode** : Au lieu de changer le style ligne par ligne, nous avons utilisé `classList.toggle('open')`.
- **Acquis** : Bonne pratique de séparation du code : le CSS gère l'animation (transition, rotation), le JS gère uniquement l'état (ouvert/fermé).

Créez un menu glissant

Crée un menu qui s'ouvre/replie au clic:



`classList.toggle('open')` :. Au lieu de modifier manuellement le style (`style.display = 'block'`), on ajoute juste une étiquette (une classe) et on laisse le CSS gérer l'affichage et la rotation de la flèche.

Structure HTML : Note comment j'ai englobé le tout dans une `div class="menu-container"`. Cela permet, si on a plusieurs menus plus tard, de ne pas mélanger les interactions.

```
1 <!DOCTYPE HTML>
2 <html lang="fr">
3 <head>
4   <meta charset="utf-8">
5   <title>Menu Glissant</title>
6   <style>
7     /* Style de base pour rendre le titre cliquable */
8     .menu-header {
9       cursor: pointer;
10      font-weight: bold;
11      user-select: none; /* Empêche la sélection du texte au double-clic */
12    }
13
14    /* Style de la flèche */
15    .arrow {
16      display: inline-block;
17      transition: transform 0.2s; /* Petite animation sympa */
18    }
19
20    /* Par défaut, la liste est cachée */
21    ul {
22      display: none;
23      list-style-type: none; /* Enlève les puces */
24      margin-top: 5px;
25      padding-left: 20px;
26    }
27
28    /* --- CLASSE UTILITAIRE --- */
29    /* Quand le menu a la classe "open", on change le style */
30
31    /* 1. On affiche la liste */
32    .menu-container.open ul {
33      display: block;
34    }
35
36    /* 2. On tourne la flèche (CSS Transform) */
37    .menu-container.open .arrow {
38      transform: rotate(90deg);
39    }
40  </style>
41 </head>
42 <body>
43
44 <div id="monMenu" class="menu-container">
```

```
46   <div class="menu-header">
47     <span class="arrow">></span> Sweeties (click me)!
48   </div>
49
50   <ul>
51     <li>Cake</li>
52     <li>Donut</li>
53     <li>Honey</li>
54   </ul>
55
56 </div>
57
58 <script>
59   // 1. Sélection des éléments
60   // On sélectionne le conteneur principal ou le titre
61   let menu = document.getElementById('monMenu');
62   let header = menu.querySelector('.menu-header');
63
64   // 2. Gestionnaire d'événement
65   header.addEventListener("click", function() {
66
67     // 3. Action : Basculer la classe
68     // 'toggle' ajoute la classe si elle n'est pas là, et l'enlève si elle est là.
69     // C'est beaucoup plus simple que de faire des "if/else" manuels !
70     menu.classList.toggle('open');
71   });
72 </script>
73
74 </body>
75 </html>
```

4. Manipulation du DOM ("Bouton de fermeture")

- **Objectif** : Ajouter dynamiquement un bouton [x] sur une liste de messages existants.

matisse kra
bts sio

- **Méthode** : Boucle `for` sur les éléments, création via `document.createElement`, insertion via `prepend`, et suppression via `remove()`.
- **Acquis** : Manipulation avancée de l'arbre DOM (ajout/suppression de nœuds).

Le problème : Nous avons 3 articles (`.pane`). Nous devons ajouter un bouton [x] à l'intérieur de chacun d'eux. Quand on clique sur ce bouton, l'article entier doit disparaître.

Problème CSS : Pour que le bouton soit "dans le coin supérieur droit", il faut utiliser le positionnement absolu. Or, pour que le bouton se positionne par rapport à l'article (et non la page entière), l'article doit être en position "relative".

- `document.createElement('button')` : Au lieu d'écrire le HTML à la main, on demande au JavaScript de fabriquer une balise. C'est plus dynamique (si tu as 1000 messages, le script marchera pareil sans effort supplémentaire).
- `pane.prepend(btn)` : Cette méthode insère le bouton à l'intérieur de la div `.pane`, mais tout au début (avant le titre `<h3>`).
- La fermeture : Dans la fonction `onclick`, on appelle `pane.remove()`. Comme la variable `pane` est définie dans la boucle, la fonction "se souvient" (grâce à ce qu'on appelle une closure) de quel panneau précis il s'agit.

matisse kra

bts sio

le css

```
6 <style>
7   body {
8     margin: 10px auto;
9     width: 470px;
10  }
11  h3 {
12    margin: 0;
13    padding-bottom: .3em;
14    font-size: 1.1em;
15  }
16  p {
17    margin: 0;
18    padding: 0 0 .5em;
19  }
20
21  /* --- MODIFICATIONS CSS --- */
22  .pane {
23    background: #edf5e1;
24    padding: 10px 20px 10px;
25    border-top: solid 2px #c4df9b;
26    margin-bottom: 10px; /* Ajouté pour espacer les messages */
27
28    /* INDISPENSABLE : Définit le repère pour le bouton "absolute" */
29    position: relative;
30  }
31
32  .remove-button {
33    /* Le style fourni + le positionnement */
34    font-size: 110%;
35    color: darkred;
36    width: 24px;
37    height: 24px;
38    border: none;
39    background: transparent;
40    cursor: pointer;
41
42    /* Positionnement dans le coin */
43    position: absolute;
44    top: 0;
45    right: 0;
46  }
47 </style>
```

le javascript

```
<script>
// 1. On sélectionne tous les éléments "pane"
// querySelectorAll renvoie une liste (NodeList) qu'on peut parcourir
let panes = document.querySelectorAll('.pane');

// 2. On parcourt chaque panneau un par un
for(let pane of panes) {
  // On crée la balise <button>
  let btn = document.createElement('button');
  // On lui donne sa classe CSS
  btn.className = 'remove-button';
  // On écrit le texte dedans
  btn.innerHTML = '[x]';

  // --- Gestion de l'événement ---
  // On assigne la fonction qui s'exécutera au clic
  btn.onclick = function() {
    // 'pane' est la variable de notre boucle actuelle.
    // La méthode remove() supprime l'élément du HTML.
    pane.remove();
  };

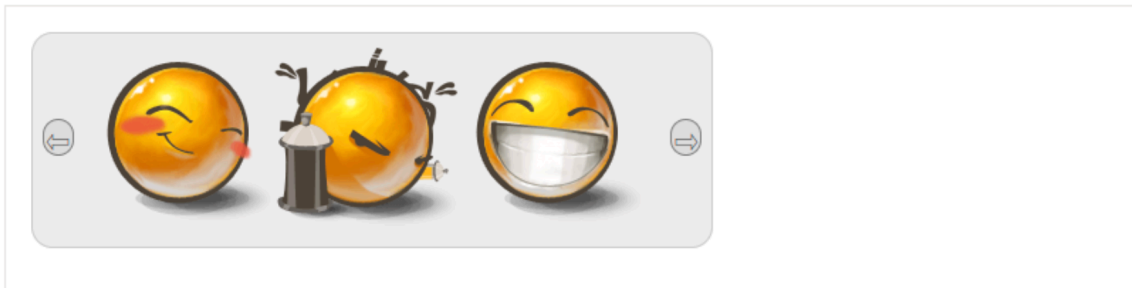
  // --- Insertion ---
  // 'prepend' ajoute l'élément comme PREMIER enfant de la div .pane
  // (append l'aurait mis à la fin)
  pane.prepend(btn);
}
</script>
```

5. Algorithme Complexe ("Le Carrousel")

- **Objectif** : Créer un ruban d'images défilant horizontalement.
- **Méthode** : Utilisation d'un conteneur avec `overflow: hidden` et modification de la `margin-left` du ruban.
- **Acquis** :
 - Compréhension que la structure HTML/CSS est vitale pour certains scripts JS.
 - Utilisation de fonctions mathématiques (`Math.min`, `Math.max`) pour gérer les limites du défilement.

Carrousel

Créez un « carrousel » – un ruban d'images que l'on peut faire défiler en cliquant sur des flèches.



P.S. Pour cette tâche, la structure HTML/CSS représente en fait 90% de la solution.

pour que ça marche, il nous faut une hiérarchie spécifique :

1. Les Variables de Configuration

- `position = 0` : Au début, le ruban est à sa position d'origine.
- Pourquoi 130 ? C'est défini dans ton CSS (`width: 130px`).

2. Le Mécanisme de Mouvement (`margin-left`)

En CSS, pour déplacer un élément vers la gauche, on peut utiliser une marge négative.

- `margin-left: 0px` : On voit les images 1, 2, 3.
- `margin-left: -130px` : La bande recule de 130px vers la gauche. L'image 1 disparaît "dans le mur", l'image 2 prend sa place.
- `margin-left: -390px` : La bande a reculé de 3 images (3 x 130). On voit maintenant les images 4, 5, 6.

L'objectif du script est donc de calculer ce nombre négatif.

3. Le Bouton "Suivant" (⇒) : On recule la bande

Quand on clique sur suivant, on veut voir les images de droite. Donc la bande doit glisser vers la gauche (valeur négative).

JavaScript

Pourquoi `Math.max` ? C'est contre-intuitif avec des nombres négatifs.

- Imaginons que la limite (fin du ruban) est -910px.
- Si le calcul nous donne -1170px (on est allé trop loin), on doit corriger.

matisse kra
bts sio

- Mathématiquement, -910 est plus grand que -1170 (car plus proche de 0).
- Donc `Math.max(-1170, -910)` renvoie -910. On reste bloqué à la fin.

4. Le Bouton "Précédent" (⏮) : On avance la bande

Quand on clique sur précédent, on veut ramener la bande vers la droite (vers 0).

JavaScript

Pourquoi `Math.min` ?

- On veut s'arrêter à 0.
- Si le calcul nous donne +390 (on a dépassé le début).
- `Math.min(390, 0)` renvoie 0.
- Cela force la bande à ne jamais avoir de marge positive.

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="utf-8">
5   <title>Carrousel JS</title>
6   <style>
7     body {
8       padding: 10px;
9       background: #f5f5f5;
10    }
11
12    /* Le conteneur global pour aligner flèches et images */
13    .carousel {
14      display: flex;
15      align-items: center; /* Centre verticalement */
16      gap: 10px; /* Espace entre flèches et images */
17      padding: 20px;
18      background: white;
19      border-radius: 10px;
20      width: fit-content;
21    }
22
23    /* LES 90% DE LA SOLUTION SONT ICI : */
24    .gallery {
25      width: 390px; /* 130px * 3 images */
26      overflow: hidden; /* Cache ce qui dépasse ! */
27      border: 2px solid #333;
28    }
29
30    ul {
31      height: 130px;
32      width: 999px; /* Très large pour tout contenir sur une ligne */
33      margin: 0;
34      padding: 0;
35      list-style: none;
36      font-size: 0;
37
38      /* Pour l'animation fluide */
39      transition: margin-left 0.5s ease-in-out;
40    }
41
42    ul li {
43      display: inline-block;
44    }
45  </style>
46 </html>
```

matisse kra
bts sio

```
46   ul img {
47     width: 130px;
48     height: 130px;
49     display: block;
50   }
51
52   .arrow {
53     padding: 10px;
54     background: #ddd;
55     border-radius: 50%;
56     border: 1px solid #gray;
57     font-size: 24px;
58     cursor: pointer;
59   }
60   .arrow:hover { background: #ccc; }
61 </style>
62 </head>
63
64 <body>
65
66   <div id="carousel" class="carousel">
67
68     <button class="arrow prev"></button>
69
70     <div class="gallery">
71       <ul id="ruban">
72         <li></li> Add an "alt" attribute to this image.
73         <li></li> Add an "alt" attribute to this image.
74         <li></li> Add an "alt" attribute to this image.
75         <li></li> Add an "alt" attribute to this image.
76         <li></li> Add an "alt" attribute to this image.
77         <li></li> Add an "alt" attribute to this image.
78         <li></li> Add an "alt" attribute to this image.
79         <li></li> Add an "alt" attribute to this image.
80         <li></li> Add an "alt" attribute to this image.
81         <li></li> Add an "alt" attribute to this image.
82       </ul>
83     </div>
84
85     <button class="arrow next"></button>
86
87
88
89 <script>
90   // --- 1. CONFIGURATION ---
91   let width = 130; // largeur d'une image
92   let count = 3; // nombre d'images visibles à la fois
93   let list = document.getElementById('ruban');
94   let listElems = list.querySelectorAll('li');
95
96   let position = 0; // position actuelle du ruban (en pixels)
97
98   // Sélection des flèches
99   let prevBtn = document.querySelector('.prev');
100  let nextBtn = document.querySelector('.next');
101
102  // --- 2. LOGIQUE SUIVANT ---
103  nextBtn.onclick = function() {
104    // On décale vers la gauche (négatif) de la largeur de 3 images
105    position -= width * count;
106
107    // Limite : ne pas aller trop loin
108    // La position minimum est : -(longueur totale - largeur visible)
109    // Math.max prend le plus grand des deux, donc si 'position' est trop petite (-2000),
110    // il prendra la limite calculée.
111    position = Math.max(position, -width * (listElems.length - count));
112
113    list.style.marginLeft = position + 'px';
114  };
115
116  // --- 3. LOGIQUE PRÉCÉDENT ---
117  prevBtn.onclick = function() {
118    // On décale vers la droite (positif)
119    position += width * count;
120
121    // Limite : ne pas dépasser 0 (le début)
122    // Math.min prend le plus petit. Si position vaut 390 (trop grand), il prendra 0.
123    position = Math.min(position, 0);
124
125    list.style.marginLeft = position + 'px';
126  };
127 </script>
```

III. Conclusion

Cet atelier a permis de passer de la théorie du langage JavaScript (variables, boucles) à son application concrète dans le navigateur. Nous avons appris que l'interactivité repose sur l'écoute d'événements et la manipulation du DOM en conséquence. La maîtrise de `addEventListener` et de l'objet `event` est essentielle pour le développement front-end moderne.